U-HAP

Unified Authorization over Heterogeneous Policies with Efficient Multi-Resource Verification in Kubernetes

JCSSE 2026 ▪ Conference Paper Presentation

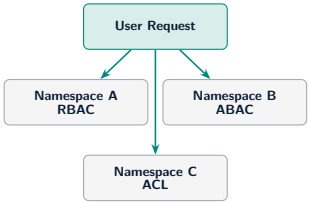
Krittapak Jairak Singha Junchan Phisitphon Pruksorranan

Advisor: Asst. Prof. Dr. Somchart Fugkeaw

School of ICT, SIIT, Thammasat University, Thailand

Motivation: The Kubernetes Authorization Problem

Modern K8s deployments are *multi-model*



Separate checks, incompatible semantics

RBAC Role-Based: grant access by assigning *roles* to users

ABAC Attribute-Based: grant access via *attribute conditions*

ACL Access Control List: explicit *named list* of allowed users

Three Key Challenges

- **Semantic fragmentation** — RBAC, ABAC, ACL use incompatible rule languages and evaluation semantics
- **Non-deterministic conflicts** — Implicit ordering across models yields inconsistent deny/allow decisions
- **Runtime inefficiency** — Repeated policy scans evaluate irrelevant rules for every request

Even with SSO handling authentication, *authorization across multiple resources remains fragmented, slow, and inconsistent.*

Our Solution: U-HAP

Core Idea

Shift expensive reasoning *offline*. Compile all policy models into **resource–action indexed artifacts** $C_{\langle n,r,a \rangle}$. Every request-time authorization reduces to an O(1) index lookup followed by cheap, model-specific index evaluation.

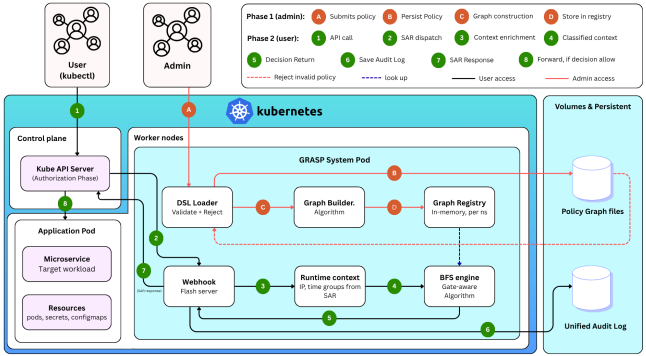
Three-Phase Architecture

- **Phase 1** Setup & trust configuration (deploy-time)
- **Phase 2** Compilation: DSL parse → DAG → indexed artifacts (policy-change time)
- **Phase 3** Request eval: O(1) lookup → pruning → index eval (every SAR — SubjectAccessReview)

Key Contributions

- **Universal semantic graph** capturing RBAC, ABAC, and ACL in a unified DAG (Directed Acyclic Graph) with deny-override semantics
- **Hash consing**: structurally identical predicates share one node — 11 nodes reduced to 8 in our 4-policy example
- **Two-level pruning**: policy-type pruning skips inactive model classes; token-driven pruning limits evaluation to relevant rules
- **Deterministic conflict resolution**: deny-overrides-all, always, across all models

System Architecture: Three-Phase Design



Phase 1: Setup (deploy-time)

- Load & validate DSL policies
- Init webhook; zero request-time cost

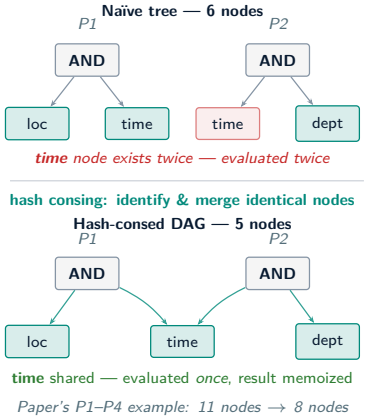
Phase 2: Compilation (per policy change)

- Parse DSL $\rightarrow$ hash-consed DAG
- Compile $C_{\langle n,r,a \rangle}$ indices per triple
- Store artifacts in registry

Phase 3: Request-time (every SAR)

- O(1) lookup on (n, r, a)
- Two-level pruning skips inactive models
- Eval: hash set / bit-vector / gates
- Return ALLOW/DENY + audit log

Compilation: Hash Consing and Indexed Artifacts



What Hash Consing Gives You

- Identical sub-expressions across policies share **one DAG node**
- Each node evaluated *at most once* per request (memoized)
- Structural equality check — not string comparison
- Cost grows sub-linearly as policies share predicates

Compiled Indexed Artifacts $C_{\langle n,r,a \rangle}$

For each `(namespace, resource, action)` triple:

I_{deny}	deny candidate set (token-pruned hash set)
I_{acl}	ACL user/group hash set (O(1) membership)
b_{rbac}	RBAC bit-vector (one AND with b_{user})
I_{abac}	ABAC gate list, cost-sorted, key-pruned

Runtime order: Cache → Deny → ACL → RBAC → ABAC → Default Deny

The DAG is a compile-time artifact only. No graph traversal at runtime — pure index lookup.

Two-Level Pruning Strategy

Level 1 — Policy-Type Pruning

Skip entire model classes that cannot produce a decision for this request.

- If no deny rules exist for `(n,r,a)`: skip deny phase
- If no RBAC policies apply: skip bit-vector AND
- If caller has no attributes: skip ABAC gate list

Level 2 — Token-Driven Pruning

Restrict evaluation to rules matching the caller's token.

- **ACL**: hash-set membership — O(1), no scan
- **RBAC**: b_{user} & $b_{\text{rbac}} \neq 0$ (single bitwise AND)
  Each bit = one role. User has admin(bit 2)+viewer(bit 5): b_{user}=...100100, policy needs admin: b_{rbac}=...000100
  AND $\neq 0 \Rightarrow$ **ALLOW**. One operation regardless of role count.
- **ABAC**: attribute-key index narrows candidates before gate evaluation

Deny-Overrides-All Invariant

Deny is always checked first. If any deny candidate matches the caller's token and request context, the decision is DENY — regardless of any allow rules in ACL, RBAC, or ABAC.
This invariant is enforced at compile time and cannot be bypassed.

Runtime Evaluation Pipeline

Default DENY if no rule matches

Evaluation Setup

Hardware and Implementation

- AMD Ryzen 9 7945HX (16C/32T), 32 GB DDR5-4800, CachyOS Linux
- U-HAP: Python 3.11 (in-memory evaluation)
- Baseline: conventional SSO-based system performing sequential policy scanning

Why In-Memory Benchmarks?

- Isolates *pure algorithm cost* from network and framework overhead
- Reveals true compilation gains: hash-consed DAG vs. sequential scan
- Caching effect visible directly without HTTP round-trip noise

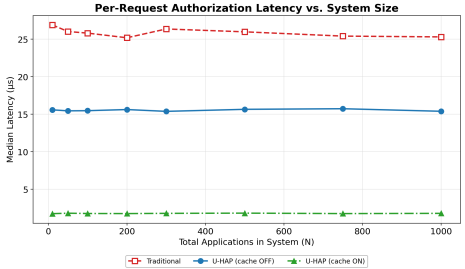
Measurement Methodology

- Median of **1,000 iterations** after 50 warm-up runs
- **In-memory policy lookup and verification only** — no network, no auth overhead
- Each namespace: 46 rules (10 RBAC, 20 ABAC, 10 ACL, 1 deny, 5 hierarchy edges)
- ABAC gates: AND/OR/ATLEAST with $\sim$50% atom sharing across rules

Baseline: SSO-Based Sequential Evaluation

- Iterates over all policy rules per request
- RBAC: repeated role resolution + hierarchy traversal at runtime
- Represents conventional authorization without compile-time indexing

Experiment 1: Policy Verification Efficiency



Latency (µs, median, 1 000 iters)

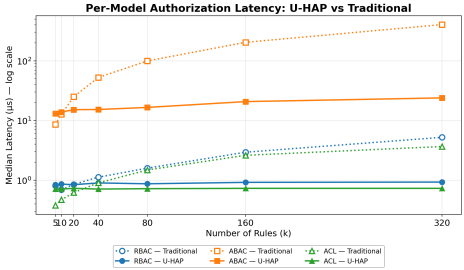
46 rules each: 10 RBAC + 20 ABAC + 10 ACL + 1 deny

N	U-HAP	+Cache	SSO
10	15.6	1.7	26.9
50	15.4	1.8	26.0
100	15.5	1.8	25.8
200	15.6	1.7	25.2
300	15.4	1.8	26.4
500	15.6	1.8	26.0
750	15.7	1.7	25.4
1000	15.4	1.8	25.3

Key Results

- **Isolation:** curves flat as N:10→1,000 — adding namespaces has *zero* cost
- **U-HAP:** ≈15.5 µs — O(1) lookup, constant vs. N
- **+Cache:** ≈1.8 µs — **14–15**× speedup over SSO
- **SSO:** ≈26 µs — 1.6–1.7× slower than U-HAP

Experiment 2: Policy Size Impact



Speedup vs. SSO baseline

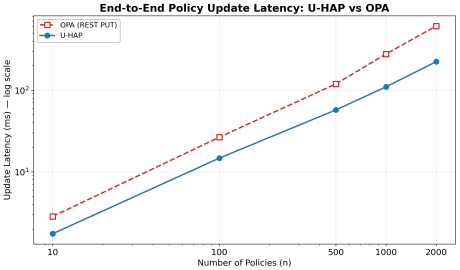
$<1\times$ = compilation cost not yet amortized

k	ABAC	RBAC	ACL
5	0.66×	0.96×	0.54×
10	0.92×	0.80×	0.66×
20	1.64×	1.01×	0.85×
40	3.44×	1.25×	1.27×
80	6.00×	1.83×	2.05×
160	9.83×	3.20×	3.58×
320	**16.9×**	**5.6×**	**5.0×**

Key Results

- **ABAC:** hash-consed DAG → **16.9**× at k=320; crossover $k\approx18$
- **RBAC:** bit-vector AND → **5.6**× at k=320; crossover $k\approx15$
- **ACL:** hash-set → **5.0**× at k=320; crossover $k\approx35$
- $<1\times$ at small k: compilation overhead not yet amortized

Experiment 3: Policy Update Latency vs. OPA



Edit-to-first-decision latency (ms)

	Total (ms)			Post-parse (ms)	
n	U-HAP	OPA	×	U-HAP	OPA
10	1.75	2.85	1.63	0.14	2.85
100	14.76	26.60	1.80	2.03	26.60
500	57.35	119.40	2.08	7.87	119.40
1000	109.99	277.32	2.52	16.27	277.32
2000	**223.41**	**610.54**	**2.73**	**31.31**	**610.54**

Key Results

- **2.73× faster** total update at n=2,000 (223 ms vs. 611 ms)
- **19.5× faster** post-parse engine path at n=2,000
- U-HAP grows linearly; OPA grows super-linearly
- 100/100 decisions verified identical (equiv. gate)

Conclusion

What We Built

U-HAP is a **compilation-driven, index-based** Kubernetes webhook authorizer unifying RBAC, ABAC, and ACL under a single semantic DAG with deterministic deny-override conflict resolution.

Key Results

- **Near-constant latency**: $\approx$1.7$\times$ vs. SSO; flat to N=1,000 namespaces
- $\sim$14$\times$ **caching speedup** on repeated requests
- **16.9**$\times$ **ABAC speedup** at k=320; RBAC 5.6$\times$, ACL 5.0$\times$
- **2.73**$\times$ **faster** policy-update vs. OPA at n=2,000
- **19.5**$\times$ **faster** post-parse path vs. OPA

Why It Works

- **Compile, don't scan** — all complexity offline; runtime is index lookup + bitwise ops only
- **Share, don't repeat** — hash consing eliminates redundant sub-expression evaluation
- **Prune early, prune deep** — two-level pruning skips entire model classes before eval

Future Work

- Reimplement in Go/Rust to eliminate Python/Gunicorn HTTP overhead
- Cross-namespace policy composition
- Dynamic policy reload without restart

Questions?

Thank you for your attention.

Singha Junchan — 6622770350@g.siit.tu.ac.th
School of ICT, SIIT, Thammasat University

Backup: Runtime Evaluation Order (Phase 3)

Evaluation Pipeline for each SAR

1. **Cache hit?** Return cached decision immediately
2. **Deny check**: token-pruned candidates from I^{deny}; any match $\Rightarrow$ `DENY`
3. **ACL**: $\text{uid} \in I_{\text{acl}}$ or $\text{groups} \cap I_{\text{acl}} \neq \emptyset$ (hash set, O(1))
4. **RBAC**: b_{user} & $b_{\text{rbac}} \neq 0$ (one bitwise AND)
5. **ABAC**: attribute-key pruned $\rightarrow$ cost-sorted $\rightarrow$ hash-consed gate eval
6. **Default Deny**: no rule matched $\Rightarrow$ `DENY`

Correctness Invariants

- **Deny-overrides-all** checked first, always
- **DAG acyclicity** verified at compile time
- **Resource-action isolation**: eval of (n, r, a) touches only $C_{\langle n,r,a \rangle}$
- **Deterministic**: same input $\Rightarrow$ same output
- **Memoization**: each DAG node evaluated at most once per request
- **Cache consistency**: invalidated on any policy change

Backup: Correctness Scenarios S1–S7

ID	Subject	Model	Resource / Verb	Context	Expected
S1	alice	ABAC	pods/prod / get	on-premise + business-hours	ALLOW
S2	alice	ABAC	pods/prod / get	remote	DENY
S3	alice	RBAC	pods/dev / get	any	ALLOW
S4	bob	ABAC	pods/prod / delete	after-hours	DENY
S5	*	Deny	secrets / delete	any	DENY
S6	charlie	ACL	pods/dev / get	any	ALLOW
S7	dave	Hierarchy	pods/prod / get	any	ALLOW

S7: dave is a Senior Developer. RBAC bit-vector encodes full transitive role closure at compile time — no runtime BFS needed.

All 7 scenarios are non-negotiable correctness gates. All pass in the U-HAP implementation.

Backup: Hash Consing — Worked Example (P1–P4)

Without Hash Consing (Naïve Tree)

Four policies P1–P4 share the predicate `time.hour` $\in$ `[9,18]` and `location = "on-premise"`, but each expands them independently.
Naïve tree: **11 nodes**.
The shared predicates are evaluated *redundantly* for each policy, even within the same request.

With Hash Consing (DAG)

Structural equality check at compile time identifies shared sub-expressions. One canonical node per unique predicate.
Hash-consed DAG: **8 nodes**.
At request time, each node is evaluated *at most once* and its result memoized. Policies referencing the same predicate re-use the cached result — zero redundant computation.

As policy sets grow and predicates are reused across rules, hash consing provides sub-linear growth in both DAG size and evaluation cost.

Backup: RBAC Bit-Vector and Role Closure

Compile-Time Transitive Closure

Role hierarchy is a DAG (acyclicity checked at compile time). Compiler computes full transitive closure for every role r:

$$b_r = \left\{ r' \mid r' \text{ is reachable from } r \right\}$$

encoded as a bit-vector. User's effective vector:

$$b_{\text{user}} = \bigvee_{r \in \text{user.roles}} b_r$$

Runtime RBAC Check

Each (n, r, a) artifact stores b_{rbac} — roles allowed for this resource/action:

$$\text{ALLOW} \iff b_{\text{user}} \;\&\; b_{\text{rbac}} \neq 0$$

One bitwise AND. No graph traversal. Independent of rule count.

Why This Eliminates Linear Scan

- Traditional RBAC (SSO baseline): iterate roles + hierarchy traversal per request — O(#roles × #rules), cost scales linearly with rule count
- U-HAP at k=320 RBAC rules: **5.6**× speedup vs. baseline
- At k=1,000+: speedup continues to grow as baseline cost scales linearly

Hierarchy traversal cost is paid *once* at compilation, not at every request.

Backup: Decision Cache

Cache Behavior

- Cache key: $(n, r, a, \text{uid}, \text{groups}, \text{attributes})$
- Cache hit returns stored ALLOW/DENY instantly
- **Invalidated** on any policy change (compilation event)
- Consistency invariant: stale decisions never served after policy update

Measured Cache Impact

In-memory benchmark (1 000 iterations, 50 warm-ups):
$\approx$**14**$\times$ **speedup** on repeated requests — cache bypasses all evaluation, returning the stored decision in one hash-table lookup.
Benefit compounds at scale: Exp1 shows the $14\times$ factor holds across $N = 1{,}000$ namespaces.

The cache is most valuable when the webhook is co-located with the API server — U-HAP is designed for exactly this deployment model.

Backup: Implementation Details

Codebase

Component	Language
DSL models + parser	Python 3.11
Graph builder + hash cons	Python 3.11
Index compiler + registry	Python 3.11
Pruning engine + evaluator	Python 3.11
Webhook handler	Flask 3.x + Gunicorn
Audit logger	Python 3.11

Deployment and Scope

- Standard K8s `SubjectAccessReview` webhook
- Deployment manifests in `deploy/`
- Test suite: S1–S7 scenario tests, hash-consing gate, role-closure, evaluator unit tests

Out of scope:

- Cross-namespace access / ClusterRole
- Dynamic policy reload without restart
- JWT validation / authentication

Backup: Policy DSL — YAML Format

Sample Policy File (namespace: prod, resource: pods)

```
namespace: prod resource: pods
rules:
# ABAC -- attribute conditions
- type: abac action: get
  predicate: "net=='on-premise' AND
time=='business-hours'"
# RBAC -- role-based access
- type: rbac role: viewer action: get
# ACL -- direct user grant
- type: acl subject: charlie action: get
# Deny -- always evaluated first
- type: deny subject: "*" action: deletecollection
# Hierarchy -- role inheritance
- type: hierarchy
  parent: senior-dev child: junior-dev
```

Five Rule Types

`deny`	Hard block; checked first, overrides any allow
`acl`	Direct uid/group grant; O(1) hash-set lookup
`rbac`	Role-based; hierarchy compiled into bit-vector
`abac`	Attribute predicate gate; hash-consed + cost-sorted
`hierarchy`	Role inheritance edge (parent $\rightarrow$ child)

Compiled Artifacts $C_{\langle n,r,a \rangle}$

I_{deny}	Deny candidate set (token-pruned hash set)
I_{acl}	ACL uid + group hash set
b_{rbac}	RBAC transitive-closure bit-vector
I_{abac}	ABAC gate list, cost-sorted

Backup: System Comparison

Feature	U-HAP	K8s RBAC	OPA	Keycloak 24
Multi-model support	**RBAC+ABAC+ ACL+Deny**	RBAC only	Policy-defined	RBAC+group+user
Deny-overrides-all guarantee	**Always, enforced**	Not explicit	Policy-defined	Not explicit
Compile-time indexing	**Full**	No	Partial (Rego partial eval)	No
Request-time complexity	**O(1) lookup**	O(rules)	O(rules)	O(rules)
K8s SAR webhook	**Native**	Built-in	Via adapter	Via adapter
Rule-density speedup (RBAC)	**5.6× at k=320**	—	—	linear scan

Backup: Limitations and Future Work

Current Limitations

- **Single-namespace**: no cross-namespace access or `ClusterRole` support
- **Restart to reload**: policy updates require webhook restart; hot-reload is not implemented
- **Authorization only**: JWT/OIDC authentication is out of scope
- **Simple deny predicates**: deny rules match on subject+action only, not attribute conditions
- **Single-process**: no multiprocessing (ThreadPoolExecutor used for batch evaluation only)

Future Directions

- **Cluster scope**: extend $C_{\langle n,r,a \rangle}$ to `ClusterRole` and cross-namespace policies
- **Incremental compilation**: re-compile only affected (n, r, a) artifacts on policy edit
- **Conditional deny**: ABAC-style attribute predicates inside deny rules
- **gRPC / Unix socket**: lower-latency transport for co-located deployment
- **Formal verification**: prove deny-overrides invariant holds for all valid DSL inputs